AgriVision Enterprise: Universal Plant Pathologist

End-to-End Deep Learning Architecture & Explainable AI (XAI) Pipeline

Technical Project Portfolio | Artificial Intelligence & Machine Learning Engineering

Executive Summary: Architected and deployed a highly scalable, multi-class Convolutional Neural Network (CNN) pipeline designed to identify agricultural diseases across diverse crop species. Originally prototyped as a 10-class MATLAB concept, the system was entirely re-engineered into a production-ready Python enterprise application. It dynamically processes over 45,000 images, maps 49 distinct disease classes without hardcoding, and renders native GUI heatmaps using Explainable AI (LIME) to establish user trust through algorithmic transparency.

> Key Achievement: Successfully eliminated the "Black Box" deep learning problem by pairing a 99.3% accuracy 3-Block CNN with Local Interpretable Model-agnostic Explanations (LIME), packaged within a standalone, OS-native desktop engine.

1. Core Engineering & Architectural Capabilities

Massive Dataset Aggregation

Engineered a recursive deep-search script to traverse and unify fragmented agricultural datasets (including PlantVillage and PlantDoc). The pipeline automatically compiled, deduplicated, and organized **45,529 high-resolution images** into a universal training structure.

Dynamic Class Mapping (Zero Hardcoding)

Eliminated brittle hardcoded labeling logic. The CNN orchestration script dynamically interrogates the dataset directory structure during the `ImageDataGenerator` flow, automatically extracting class names and generating a persistent `class_mapping.json` dictionary for runtime frontend injection.

Scalable 3-Block CNN Architecture

Designed a robust, hardware-aware CNN featuring stacked Conv2D layers (32, 64, 128 filters), Batch Normalization for rapid convergence, and targeted Dropout (25%-50%) to aggressively combat overfitting. Achieved **99.35% Training Accuracy** and **~93.20% Validation Accuracy** strictly utilizing CPU compute limitations.

Decoupled Orchestration Pipeline

Abstracted the machine learning lifecycle into a CLI-driven orchestrator (`main.py`). Modularized preprocessing, K-Nearest Neighbors (KNN) baseline training, CNN execution, and XAI generation into independent pipeline arguments for enterprise scalability.

2. Explainable AI (XAI) Integration

To bridge the trust gap between complex AI predictions and agricultural end-users, the system integrates **LIME (Local Interpretable Model-agnostic Explanations)** directly into the inference loop. Rather than merely outputting a confidence score, the engine computes visual proof of its neural pathways.

- **Feature Isolation (Mask 1):** Computes a strictly positive perturbation mask, highlighting the top 5 localized superpixels (e.g., specific rust spots or leaf lesions) that actively triggered the network's classification.
- **Decision Contrast (Mask 2):** Renders a comprehensive topological heatmap overlaying the leaf, contrasting regions that mathematically supported the prediction (Green) against regions that opposed it (Red).
- **Thread-Safe Processing:** Structured the UI callbacks to prevent main-thread freezing during heavy LIME instance permutations, ensuring a smooth UX during deep pixel analysis.

3. Native Application Frontend (CustomTkinter)

Bypassed legacy web-browser constraints and "DLL Hell" UI conflicts by engineering a standalone, native Windows desktop application utilizing Python's `CustomTkinter`.

- **Object-Oriented UI:** Constructed a responsive, class-based grid layout isolating control panels from diagnostic rendering regions.
- **Dynamic Theming:** Implemented tuple-based color mapping `("Light_Color", "Dark_Color")`, providing a real-time Light/Dark mode toggle to ensure accessibility and modern software design standards.
- **Seamless Backend Handshake:** The UI dynamically unpacks the backend's JSON mapping file upon initialization, allowing the frontend to instantly adapt if the CNN is retrained on 10, 50, or 1,000 disease classes without a single line of UI code modification.

Technical Stack & Metrics

Domain	Technologies & Libraries Utilized
Deep Learning Engine	TensorFlow 2.x, Keras, Convolutional Neural Networks (CNN)
Machine Learning & Vision	Scikit-Learn (KNN, LabelEncoder), OpenCV (cv2), NumPy, PIL
Explainable AI (XAI)	LIME (lime_image), Scikit-Image (mark_boundaries), Matplotlib
Frontend Application	CustomTkinter, Python Native OS Interfaces
Data Scale	45,529 Images | 49 Distinct Disease Classes | 75:25 Split